Day 31: Add Product to Cart

Day 31 — Add Product to Cart

1. Day Number

Day 31

2. Topic Name

Cart List and `append()`

Today you will learn how to:

- create an empty shopping cart
- check whether a product is available
- add a product to the cart using `append()`

---

3. Connection with Yesterday

Yesterday, you worked with **multiple products stored as dictionaries**.

For example, conceptually:

```
products
   |
   +--> product 1
   +--> product 2
   +--> product 3
```

Today we introduce another list:

```
products                        cart
   |                              |
   +--> Laptop                    |
   +--> Mouse       ------->       +--> Mouse
   +--> Keyboard
```

The important idea is:

> **Products represent what the store has. Cart represents what the customer selected.**

Today we will add only **one product** to the cart.

---

4. Important Topics

The main Python concepts today are:

- `list`
- `dictionary`
- `append()`
- `if` condition
- simple product selection

- function

The new concept is mainly:

```
append()
```

---

5. Foundational Notes

What is a cart?

A shopping cart can simply be represented as a Python list.

Initially, the customer has selected nothing.

Therefore:

```
cart = []
```

This means:

```
cart
 |
 +-- empty
```

What happens when a product is selected?

Suppose we have a product:

```
product = {
    "name": "Mouse",
    "price": 800,
    "stock": 5
}
```

If the product is available, we want to put the **whole product dictionary** inside the cart.

Conceptually:

```
Before

cart
[]

Product
{
    name: Mouse
    price: 800
    stock: 5
}
```

After adding:

```
cart
 |
 +--> {
        name: Mouse
        price: 800
        stock: 5
      }
```

So the cart becomes a:

> **list of product dictionaries**

This connects directly with what you learned on Day 30.

What does append() do?

`append()` adds **one item to the end of a list**.

Simple example:

```
numbers = []

numbers.append(10)
```

Now:

```
numbers = [10]
```

Another append:

```
numbers.append(20)
```

Now:

```
numbers = [10, 20]
```

The important point is:

```
append() adds ONE item
```

Appending a dictionary

A dictionary can also be added to a list.

Conceptually:

```
cart.append(product)
```

means:

```
Take this product dictionary
        |
        v
```

```
      product
        |
        v
Add it to the cart list
```

Result:

```
cart
 |
 +--> product dictionary
```

Why check stock first?

Imagine:

```
Mouse stock = 5
```

The product is available.

Therefore:

```
stock > 0
    |
   True
    |
    v
Add to cart
```

But suppose:

```
Mouse stock = 0
```

Then:

```
stock > 0
    |
   False
    |
    v
Do not add
```

This prevents customers from adding an unavailable product.

---

6. Easy Example

Before working with products, consider a simpler example.

Suppose you have a list of fruits:

```
basket = []
fruit = "Apple"
```

You can add the fruit using:

```
basket.append(fruit)
```

Then:

```
Before:

basket = []

After:

basket = ["Apple"]
```

Now imagine fruit availability:

```
Apple quantity = 3
```

You could think:

```
Is quantity > 0?

Yes
 |
 v
Add Apple to basket
```

This is exactly the same idea we will use with products.

---

7. Problem Statement

Create a Python program that:

1. Creates one product dictionary containing:
    - `name`
    - `price`
    - `stock`
2. Creates an empty cart list.
3. Checks whether the product stock is greater than `0`.
4. If stock is greater than `0`:
    - add the product dictionary to the cart using `append()`
5. If stock is `0`:
    - do not add the product to the cart
6. Print the final cart.

Example product:

```
name  = Keyboard
price = 1500
```

```
stock = 3
```

Expected flow:

```
Create product
      |
      v
Create empty cart
      |
      v
Check product stock
      |
      v
Is stock > 0?
     / \
   Yes  No
    |    |
    v    v
 Append  Don't add
    |
    v
Print cart
```

---

8. Concepts Used

Dictionary

Stores information about one product.

```
Product
├── name
├── price
└── stock
```

List

Stores products selected by the customer.

```
cart = []
```

Eventually:

```
cart
 |
 +--> product dictionary
```

---

append()

Adds one product to the cart.

Concept:

```
cart
  +
product
  |
  v
cart.append(product)
```

---

if

Checks whether the product is available.

Conceptually:

```
if stock > 0
    add product
```

Dictionary Access

To check stock, you need to access the product's `"stock"` value.

You learned this earlier when working with dictionaries.

Think:

```
product
 |
 +-- name
 +-- price
 +-- stock  <-- we need this
```

---

9. Thought Process

When solving this problem, think in small steps.

Step 1 — What represents the product?

A dictionary.

```
product
 |
 +-- name
 +-- price
 +-- stock
```

Step 2 — What represents the shopping cart?

A list.

Initially:

```
cart = empty
```

Step 3 — What condition should be checked?

Ask:

```
Is stock greater than 0?
```

There are two possibilities.

```
          stock > 0?
            /   \
          Yes    No
           |      |
           v      v
       Add item   Skip
```

Step 4 — How do we add the product?

Use:

```
append()
```

Conceptually:

```
cart.append(product)
```

Step 5 — What should the function do?

A simple function could receive:

```
product
cart
```

and perform:

```
check stock
     |
     v
add product if available
```

You do not need complicated logic yet.

10. Pseudocode

```
START

CREATE product dictionary
    name
    price
    stock
```

```
CREATE empty cart list

DEFINE function to add product to cart

    CHECK product stock

    IF stock is greater than 0
        APPEND product to cart

    OTHERWISE
        do not add product

CALL the function

PRINT cart

END
```

Notice that the pseudocode describes the **logic**, not exact Python syntax.

---

11. Suggested Solving Approach — Functional Approach

Use a small function for the cart operation.

Conceptually:

```
product
   |
   |
   v
+----------------------+
| add_to_cart()        |
|                      |
| check stock          |
|                      |
| if available         |
|     add to cart      |
+----------+-----------+
           |
           v
          cart
```

A possible function idea is:

```
add_to_cart(product, cart)
```

Its responsibility should be only:

```
Check product availability
        +
Add available product to cart
```

This is better than putting everything directly into one long program because later your cart logic can grow.

For example, future days may introduce:

```
add_to_cart()
remove_from_cart()
calculate_total()
checkout()
```

So you are slowly building toward a small shopping-cart application.

---

12. Easy Edge Cases

Edge Case 1 — Stock is 0

Example:

```
Product:
name  = Mouse
price = 800
stock = 0
```

Condition:

```
0 > 0
```

Result:

```
False
```

Therefore:

```
cart = []
```

The product should **not** be added.

Edge Case 2 — Empty Cart

Initially:

```
cart = []
```

This is completely valid.

An empty cart simply means:

> The customer has not selected any available product yet.

If an available product is added:

```
[]
 |
```

```
 | append product
 v
[product]
```

---

13. Expected Input

For this exercise, you do not need keyboard input using `input()`.

You can define the product directly.

For example:

```
Product:

name  = "Keyboard"
price = 1500
stock = 3
```

Initial cart:

```
[]
```

---

14. Expected Output

If:

```
stock = 3
```

the product should be added.

Conceptual output:

```
Cart:
[
    {
        name: Keyboard,
        price: 1500,
        stock: 3
    }
]
```

If:

```
stock = 0
```

the product should not be added.

Expected cart:

```
Cart:
[]
```

The exact Python dictionary formatting may look slightly different when printed, and that is fine.

---

15. Hint Only

Try building your solution around these ideas:

```
product = {...}

cart = []

function(product, cart):

    if product's stock > 0:
        cart._________(product)
```

Think about:

> Which list method did we learn today that adds **one item** to a list?

That is the key to Day 31.

Day 31 mental model

```
Product Dictionary
      |
      v
Check Stock
      |
      v
 stock > 0?
   /     \
 Yes      No
  |        |
  v        v
append    skip
  |
  v
Cart List
```

Do not worry about quantities, reducing stock, removing products, or calculating totals yet. For today, focus only on:

```
Dictionary
   +
Stock check
   +
append()
   =
Add one available product to cart
```